1. How we can connect Armstrong's Inference rules to derive new functional dependencies from the given one

Answer :

Armstrong's Inference Rules are not meant to be used in isolation. The key idea is to connect Reflexivity, Augmentation, and Transitivity logically and systematically to derive new functional dependencies.
In simple problems, a single rule may be enough, but in complex cases, we often:

- Apply Augmentation to reshape dependencies
- Use Transitivity to chain them
- Combine results using Union or Decomposition

A functional dependency (FD) is a constraint between two sets of attributes in a relation.
It is denoted as:
$X \rightarrow Y$
This means: *If two tuples agree on attributes X, they must also agree on attributes Y.*
Reflexivity Rule
If $Y \subseteq X$, then:
$X \rightarrow Y$
This is also called a trivial functional dependency.
Example:
If X = (A, B), then:

- $AB \rightarrow A$
- $AB \rightarrow B$

Augmentation Rule
If $X \rightarrow Y$, then:
$XZ \rightarrow YZ$ (for any attribute set Z)
This means we can add attributes to both sides without affecting validity.
Example:
If $A \rightarrow B$, then:

- $AC \rightarrow BC$

Transitivity Rule
If:

- $X \rightarrow Y$
- $Y \rightarrow Z$

Then:
$X \rightarrow Z$
This allows us to chain dependencies.
Example:
If $A \rightarrow B$ and $B \rightarrow C$, then:

- $A \rightarrow C$

Connecting Armstrong's Rules
The real power of Armstrong's rules comes from connecting multiple rules step by step to derive new dependencies.
Simple Connection
Given:

1. $A \rightarrow B$
2. $B \rightarrow C$

Goal: Derive A → C
Solution:

- From A → B and B → C
- Apply Transitivity

A → C
Using Augmentation + Transitivity
Given:

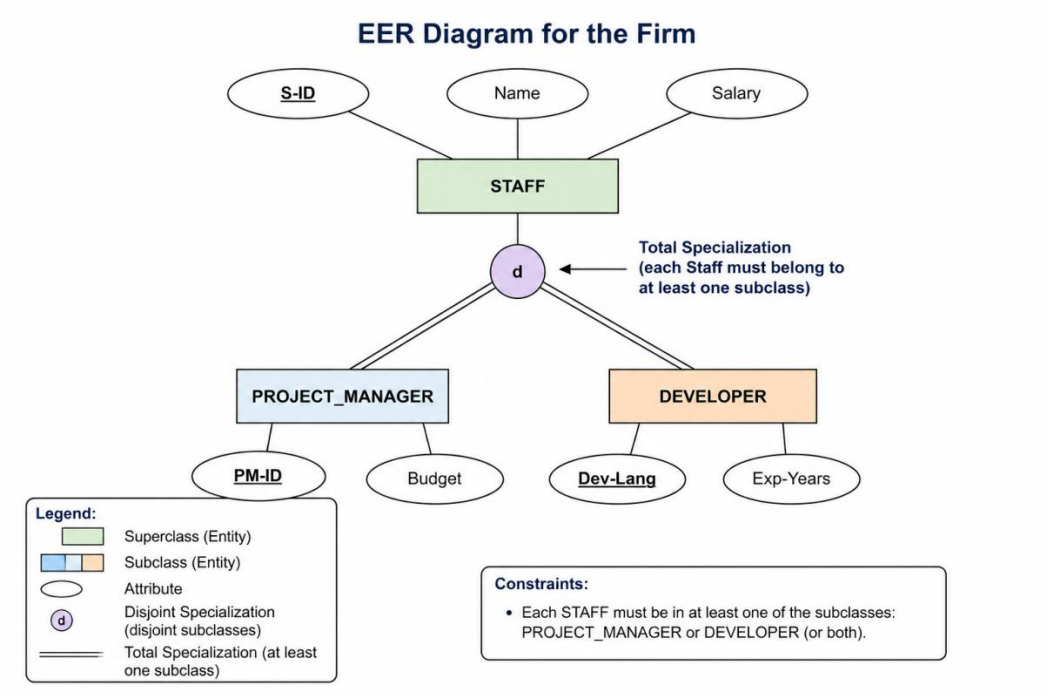
1. A → B
2. B → C

Goal: Derive AC → C
Steps:

1. A → B
2. Apply Augmentation with C:
   AC → BC
3. From B → C
4. Apply Transitivity:
   AC → C

2. Draw an EER diagram for the firm were Staff is the superclass that contains (S-ID, Name, Salary) and Staff's are specialized in to Project Manager and Developers ensuring each staff should be in at least one of the specialized subclass

Answer :



Superclass: STAFF

- The main entity is STAFF.
- It has three attributes:

- S-ID (Primary Key – uniquely identifies each staff member)
- Name
- Salary

This means every employee in the firm is first stored as a STAFF member.

Specialization (Subclassing)

- The STAFF entity is specialized into two subclasses:
  1. PROJECT_MANAGER
  2. DEVELOPER

This is called specialization in EER modeling.

Subclasses and Their Attributes

a) PROJECT_MANAGER

- Attributes:
  - PM-ID (can be considered as identifier)
  - Budget
- Represents staff who manage projects.

b) DEVELOPER

- Attributes:
  - Dev-Lang (programming language known)
  - Exp-Years (years of experience)
- Represents staff who do development work.

Total Specialization Constraint

- The double line from STAFF to the specialization circle indicates total specialization.
- Meaning:
  Every STAFF member must belong to at least one subclass
  (either PROJECT_MANAGER or DEVELOPER or both).

Disjoint Constraint (d)

- The circle is marked with "d", which means disjoint specialization.
- Meaning:
  A staff member can belong to only one subclass at a time
  (either PROJECT_MANAGER or DEVELOPER, not both).

Overall Meaning

- Every employee in the firm:
  - Must be either a Project Manager or a Developer
  - Cannot be both at the same time
- Common details (ID, Name, Salary) are stored in STAFF
- Role-specific details are stored in their respective subclasses

3. Differentiate B and B+ tree

A B-Tree is a self-balancing search tree where:

- Data (records) and keys are stored together in all nodes (internal + leaf).
- It maintains sorted order and allows search, insert, delete in O(log n) time.

A B+ Tree is an advanced version of B-Tree where:

- Only keys are stored in internal nodes
- Actual data is stored only in leaf nodes
- Leaf nodes are linked sequentially (important for range queries)

Feature	B-Tree	B+ Tree
Data Storage	In all nodes	Only in leaf nodes
Internal Nodes	Store keys + data	Store only keys
Leaf Nodes	Not necessarily linked	Always linked

Search Path	May end at internal node	Always ends at leaf
Range Queries	Slower	Very efficient
Redundancy	No duplication of keys	Keys repeated in internal nodes
Disk Access	More	Less (optimized)
Sequential Access	Poor	Excellent

B-Tree Example (order 3)

```
      [20]
     /   \
[10,15]   [25,30]
```

Data can be found in any node

B+ Tree Example (order 3)

```
      [20]
     /   \
  [10,15] [25,30]
```

Leaf nodes (linked):

[10,15] <-> [20,25,30]

Internal nodes = only keys

Leaf nodes = actual data + linked list

B-Tree

- Good for general searching
- Data stored everywhere

B+ Tree

- Best for database indexing
- Supports fast searching + efficient range queries
- Used in real DBMS systems (like MySQL, Oracle, etc.)

4. Differentiate DTD and XML schema

DTD (Document Type Definition)
Defines the structure of an XML document (what elements and order are allowed).
XML Schema (XSD)
Defines both structure + data types + constraints of an XML document.

- DTD
  - Uses its own non-XML syntax
  - Harder to read and write
- XML Schema
  - Written in XML format
  - Easy to understand and extend

Feature	DTD	XML Schema (XSD)
Syntax	Non-XML	XML-based
Data types	Limited	Rich data types
Constraints	Weak	Strong
Namespaces	Not supported	Supported
Extensibility	No	Yes
Readability	Less	More
Validation power	Low	High

DTD Example

```
<!DOCTYPE employee [
 <!ELEMENT employee (name, age)>
 <!ELEMENT name (#PCDATA)>
 <!ELEMENT age (#PCDATA)>
]>
```

Here, age is just text (no validation of number)

XML Schema Example

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
 <xs:element name="employee">
  <xs:complexType>
   <xs:sequence>
    <xs:element name="name" type="xs:string"/>
    <xs:element name="age" type="xs:integer"/>
   </xs:sequence>
  </xs:complexType>
 </xs:element>
</xs:schema>
```

Here, age must be an integer

- DTD
  - Simple XML validation
  - Older systems
- XML Schema
  - Modern applications
  - When data validation is important

5. A relation R(A,B,C) can be decomposed into R1(A,B),R2(B,C) and R3(A,C). How can you tell whether this represents join dependency?

A join dependency on a relation R(A, B, C) means:
The original relation R can be reconstructed exactly by joining its decomposed relations.
$R = R1 \bowtie R2 \bowtie R3$

Given Decomposition

- R1(A,B)
- R2(B,C)
- R3(A,C)

Take a sample relation R
Decompose into R1,R2,R3R_1, R_2, R_3R1,R2,R3
Join them back
Compare with original R:

- If same → Join Dependency exists
- If extra tuples appear → No Join Dependency

Step 1: Original Relation R

A	B	C
1	2	3
1	2	4

Step 2: Decompose

R1(A,B)

A	B
1	2

R2(B,C)

B	C
2	3
2	4

R3(A,C)

A	C
1	3
1	4

Step 3: Join

R1⋈R2⋈R3

A	B	C
1	2	3
1	2	4

Same as original R

It is a join dependency if joining R1, R2, and R3 always gives exactly R without extra tuples.

6. Explain Lock Granularity

Lock granularity refers to the size of the data item that is locked during transaction execution. It can range from database level to attribute level. Coarse granularity reduces overhead but decreases concurrency, while fine granularity increases concurrency but adds overhead. Choosing the appropriate granularity is important for efficient database performance.

Levels of Lock Granularity

Locks can be applied at different levels:

1. Database level – entire database is locked
2. Table level – whole table is locked
3. Page/Block level – a group of rows is locked
4. Row (Tuple) level – individual row is locked
5. Attribute level – specific column value is locked (rare)

Type	Description	Advantage	Disadvantage
Coarse (Table/DB)	Large data locked	Less overhead	Low concurrency
Fine (Row/Attribute)	Small data locked	High concurrency	More overhead

Consider a table: STUDENT

ID	Name	Marks
1	A	80
2	B	75
3	C	90

Case 1: Table-Level Lock (Coarse Granularity)

- Transaction T1 updates Marks of ID = 1
- Entire STUDENT table is locked

Result:

- T2 cannot access any row in the table
  Poor concurrency

Case 2: Row-Level Lock (Fine Granularity)

- T1 locks only row where ID = 1

Result:

- T2 can still access rows with ID = 2 and 3
  Better concurrency

Factor	Coarse Granularity	Fine Granularity
Overhead	Low	High
Concurrency	Low	High
Complexity	Simple	Complex

7. Distributed transactions happening in site X and site Y, where transaction T updates both sites. Explain the various steps of Two Phase Protocol and analyzes what happens if one site fails during the commit phase

Two-Phase Commit Protocol (2PC) in Distributed Transactions

Introduction

In a distributed database system, a single transaction may execute across multiple sites. Consider a transaction T that updates data at Site X and Site Y. It is essential to maintain atomicity, meaning:

Either both sites commit the transaction or both sites abort it.

To achieve this, the Two-Phase Commit Protocol (2PC) is used. It is a widely used protocol that ensures all participating sites in a distributed transaction agree on a common decision (commit or abort).

Basic Concept of 2PC

In 2PC, one site acts as the Coordinator and the others act as Participants (Cohorts).

- Let Site X be the Coordinator
- Let Site Y be the Participant

The protocol proceeds in two phases:

1. Prepare Phase (Voting Phase)
2. Commit Phase (Decision Phase)

Phase 1: Prepare Phase (Voting Phase)

In this phase, the coordinator asks all participating sites whether they are ready to commit the transaction.

Steps:

1. After completing its part of transaction T, the Coordinator (Site X) sends a *PREPARE* message to Site Y.
2. The Participant (Site Y):
    - Executes its part of the transaction
    - Writes all changes to a log (on stable storage)
    - Ensures that it can commit without errors
3. Site Y then sends its vote:
    - YES (Vote-Commit) → if it is ready to commit
    - NO (Vote-Abort) → if it cannot commit

Outcome of Phase 1:

- If all participants vote YES → proceed to commit phase
- If any participant votes NO → transaction is aborted

Phase 2: Commit Phase (Decision Phase)

Based on the votes received, the coordinator takes the final decision.

Case 1: All Participants Vote YES

1. The coordinator (X):
    - Writes COMMIT in its log

    - Sends COMMIT message to Site Y
2. The participant (Y):
    - Commits the transaction
    - Releases all locks
    - Sends ACK (acknowledgment) to X
3. The coordinator completes the transaction after receiving ACK

Case 2: Any Participant Votes NO

1. The coordinator:
    - Writes ABORT in its log
    - Sends ABORT message to all participants
2. All sites:
    - Roll back changes
    - Release resources

Example

Consider transaction T transferring ₹100 from Account A (Site X) to Account B (Site Y).

Execution:

- Site X deducts ₹100 from Account A
- Site Y adds ₹100 to Account B

Using 2PC:

- Phase 1:
  X asks Y → "Can you commit?"
  Y checks and replies YES
- Phase 2:
  X sends COMMIT
  Both sites finalize changes

Thus, both updates happen successfully, maintaining consistency.

Failure Analysis During Commit Phase

Although 2PC ensures correctness, failures can occur, especially during the commit phase.

Case 1: Participant (Site Y) Fails After Voting YES

- Y has already sent YES
- But crashes before receiving COMMIT

Effect:

- Y is in an uncertain (prepared) state
- It cannot decide whether to commit or abort

Solution:

- On recovery, Y:
    - Checks its log
    - Contacts coordinator (X)
    - Waits for final decision

This leads to the Blocking Problem, where Y must wait indefinitely if X is unavailable.

Case 2: Coordinator (Site X) Fails Before Sending COMMIT

- X receives YES from Y
- But crashes before sending COMMIT

Effect:

- Y is stuck in prepared state
- Cannot commit or abort independently

Y must wait until X recovers and sends the decision

Case 3: Coordinator Fails After Sending COMMIT

- X sends COMMIT but crashes before completion

Effect:

- Some sites may have committed
- Others are waiting

Solution:

- On recovery, X:
  - Checks its log (COMMIT record exists)
  - Resends COMMIT message

Case 4: Participant Fails After Receiving COMMIT

- Y crashes after receiving COMMIT

Effect:

- On recovery:
  - Y checks log
  - Finds COMMIT record
  - Completes transaction

Problems of Two-Phase Commit Protocol

1. Blocking Problem
   - If coordinator fails, participants cannot proceed
   - They remain in uncertain state
2. High Overhead
   - Requires multiple messages and logging
3. Slow Performance
   - Two phases increase latency

8. Explain the levels of Data Abstraction

Levels of Data Abstraction in Database Systems

Introduction

In a database system, large amounts of data are stored and managed. However, not all users need to see all the details of how data is stored. To simplify interaction and improve usability, databases use data abstraction.

Data abstraction is the process of hiding complex implementation details and showing only the necessary information to users.

It helps in:

- Reducing complexity
- Improving security
- Providing different views to different users

Three Levels of Data Abstraction

Database systems use three levels of abstraction:

1. Physical Level (Internal Level)
2. Logical Level (Conceptual Level)
3. View Level (External Level)

Physical Level (Internal Level)

Definition

The physical level describes how data is actually stored in the database.

It deals with low-level details such as:

- File organization
- Indexing
- Storage structures (B+ trees, hashing)
- Record placement on disk

Key Features

- Lowest level of abstraction
- Complex and hidden from users

- Handled by database administrators (DBA)

Example

Consider a Student table:

At the physical level:

- Data may be stored in blocks on disk
- Indexed using B+ tree
- Records stored in binary format

Users do not see this complexity.

Logical Level (Conceptual Level)

Definition

The logical level describes what data is stored and relationships among data.

👉 It includes:

- Tables (relations)
- Attributes (columns)
- Constraints (keys, relationships)

Key Features

- Middle level of abstraction
- Describes the entire database structure
- Used by database designers

Example

Student table at logical level:

Student_ID	Name	Course
101	Ravi	CS
102	Anu	IT

Attributes: Student_ID, Name, Course

Primary Key: Student_ID

View Level (External Level)

Definition

The view level describes how users see the data.

Different users can have different views of the same database.

Key Features

- Highest level of abstraction
- Simplifies interaction
- Enhances security (users see only required data)

Example

From the Student table:

Admin View:

Student_ID	Name	Course

Teacher View:

| Student_ID | Name |

Student View:

| Name | Course |

Bank Database:

- Physical Level
  Data stored in files, indexed using B+ trees
- Logical Level
  Table: ACCOUNT(Account_No, Name, Balance)
- View Level
  - Customer sees: Name, Balance
  - Manager sees: All details

Advantages of Data Abstraction

1. Reduces Complexity
   Users do not need to understand storage details
2. Improves Security
   Sensitive data can be hidden
3. Data Independence
   Changes at one level do not affect others
4. Multiple Views
   Different users see different data

9. Explain the importance of DBA in establishing the smooth working of database

A **Database Administrator (DBA)** is essential for ensuring that a database system runs smoothly, securely, and efficiently.

1. Database Installation and Setup

A DBA installs and configures database systems like Oracle Database or MySQL, ensuring the system is ready for use with proper settings.

2. Database Design Support

They help in designing the database structure (tables, relationships, indexes) to ensure efficient data storage and retrieval.

3. Performance Monitoring and Tuning

DBAs continuously monitor performance and optimize queries, indexes, and memory usage to maintain high speed and responsiveness.

4. Data Security Management

They implement strong security measures such as authentication, authorization, and encryption to protect sensitive data.

5. User Access Control

A DBA assigns roles and permissions, ensuring that users access only the data they are authorized to use.

6. Backup and Recovery Planning

They create regular backups and define recovery strategies to restore data in case of failure, ensuring business continuity.

7. Data Integrity Maintenance

DBAs enforce constraints and rules to ensure data remains accurate, consistent, and reliable.

8. Disaster Recovery Management

They prepare contingency plans for unexpected disasters like system crashes, power failures, or cyberattacks.

9. Troubleshooting and Problem Resolution

When issues occur, DBAs quickly identify and fix errors to minimize downtime and disruption.

10. Database Upgrades and Patching

They apply updates and patches to improve functionality, fix bugs, and protect against vulnerabilities.

11. Storage and Capacity Planning

DBAs monitor storage usage and plan for future growth to prevent system slowdowns or crashes.

12. High Availability Management

They implement techniques like replication and clustering to ensure the database is always accessible.

13. Documentation and Standards

A DBA maintains proper documentation and establishes standards for database usage and maintenance.

14. Data Migration and Integration

They handle data transfer between systems and ensure smooth integration with other applications.

15. Support for Developers and Users

DBAs assist developers in writing efficient queries and support end-users by resolving database-related issues.

10. Explain and suggest a solution for the database system face bucket overflow in hashing.

Bucket Overflow in Hashing (Database Systems)

In a database system, **hashing** is used to store and retrieve records quickly using a **hash function** that maps a key to a specific **bucket (storage location)**.

A **bucket overflow** occurs when:

- More records are assigned to a bucket than it can hold.
- The bucket becomes full due to collisions (multiple keys mapping to the same location).

This problem reduces performance and can lead to slow searching and insertion.

Bucket Overflow Happens

1. Poor hash function causing many collisions
2. Limited bucket size (fixed storage)
3. Uneven distribution of records
4. High data insertion rate
5. Lack of proper overflow handling technique

Solutions to Bucket Overflow in Hashing

Overflow Chains (Chaining)

- When a bucket is full, extra records are stored in a linked list (overflow chain).
- Each bucket points to a chain of overflow records.

Open Addressing

- If a bucket is full, the system searches for another empty bucket using probing techniques.

Types include:

- **Linear Probing:** Check next slot one by one
- **Quadratic Probing:** Uses quadratic intervals
- **Double Hashing:** Uses a second hash function

Extendible Hashing (Dynamic Hashing)

- The system dynamically increases the number of buckets when overflow occurs.
- Directory structure is used to map keys.

Linear Hashing

- Buckets are gradually split as the file grows.
- No need for a large directory like extendible hashing.

Increasing Bucket Size

- Increase the capacity of each bucket (more storage per block).

Rehashing

- Create a new hash table with a better hash function or more buckets.

- Reinsert all records into the new structure.

11. Explain and identify the CAP properties involved in distributed database system

CAP Properties in a Distributed Database System

The **CAP Theorem** is a fundamental concept in distributed database systems. It explains the trade-offs a system must make when dealing with networked data storage.

CAP stands for:

Consistency, Availability, and Partition Tolerance

A distributed system can guarantee **only two of these three properties at the same time**.

Consistency (C)

Meaning:

All nodes in the system see the **same data at the same time**.

Explanation:

- After an update, every read operation returns the latest value.
- No stale or outdated data is allowed.

Example:

If you update your bank balance in one node, all other nodes immediately reflect the same updated balance.

Ensures accurate and reliable data

Availability (A)

Meaning:

Every request receives a response, even if some nodes are down.

Explanation:

- The system always responds (success or failure), but data may not be the latest.
- Focus is on uptime and responsiveness.

Example:

A social media app still loads posts even if one server is temporarily unavailable.

Always accessible system

Partition Tolerance (P)

Meaning:

The system continues to operate even if **network failures (partitions)** occur between nodes.

Explanation:

- Communication between servers may be broken or delayed.
- The system still functions despite this split.

Example:

Two data centers cannot communicate due to network failure, but both continue serving users.

Essential for distributed systems

CAP Trade-offs (Important Concept)

A distributed database must choose **two out of three**:

1. CP (Consistency + Partition Tolerance)

- Ensures correct data even during network failure.
- May reduce availability.

Example systems: Banking systems, financial transactions

2. AP (Availability + Partition Tolerance)

- System remains available even during failures.
- May return slightly outdated data.

Example systems: Social media, DNS systems

3. CA (Consistency + Availability)

- Works only when no network failure exists.
- Not realistic for distributed systems.

12. The banking system deals displays incorrect account balances when two users update the same record. Explain about the issue and suggest a concurrency control mechanism that deals with it

Issue: Incorrect Account Balance Due to Concurrent Updates

In a banking database system, the problem described is a classic **concurrency problem** called a **race condition (lost update problem)**.

When two users try to update the same account at the same time:

Example scenario:

- Account balance = ₹10,000
- User A withdraws ₹2,000
- User B withdraws ₹3,000

Without proper control:

1. User A reads balance = 10,000
2. User B reads balance = 10,000 (before A updates)
3. User A updates balance → 8,000
4. User B updates balance → 7,000

Final result = ₹7,000 (incorrect)

Correct balance should be: ₹5,000

This happens because updates overlap and one update overwrites the other.

Issue Identified

This is mainly caused by:

- **Lost update problem**
- **Dirty read / inconsistent read**
- Lack of synchronization between transactions

To solve this issue, the database must control how multiple transactions access the same data.

Lock-Based Concurrency Control (Best Solution)

How it works:

- A **lock** is placed on the record before updating.
- Only one transaction can modify the account at a time.

Types of locks:

- **Shared Lock (Read lock):** Multiple users can read but not write
- **Exclusive Lock (Write lock):** Only one user can write

In banking system:

- User A locks account
- User B must wait until A completes transaction

Prevents data inconsistency

Ensures correct final balance

Two-Phase Locking (2PL)

Concept:

A transaction has two phases:

1. **Growing phase:** Acquires all required locks
2. **Shrinking phase:** Releases locks after completion

Ensures serial execution of transactions
Prevents conflicts

Timestamp Ordering

How it works:

- Each transaction gets a timestamp.
- Older transaction is given priority.

Ensures correct execution order
May cause delays in high traffic systems

Optimistic Concurrency Control

How it works:

- Transactions execute without locking.
- Before commit, system checks for conflicts.
- If conflict occurs, transaction is rolled back.

Good for low-conflict systems

Best Suitable Solution for Banking System

Lock-based concurrency control with Strict Two-Phase Locking (Strict 2PL) is the most appropriate because:

- Banking requires **high accuracy**
- Prevents **lost updates**
- Ensures **serializability**
- Guarantees **consistent account balance**

13.Analyse the need of Normalization? What problem does it solve?

Need for Normalization in Database Systems

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing large tables into smaller, well-structured tables and defining relationships between them.

Reduces Data Redundancy

Without normalization, the same data may be stored multiple times in different rows or tables. Normalization removes duplicate data, saving storage space and improving efficiency.

Prevents Data Inconsistency

When the same data exists in multiple places, updating one copy and not others leads to inconsistency. Normalization ensures data is stored in one place, so updates remain consistent.

Improves Data Integrity

It ensures accuracy and reliability of data by enforcing rules like:

- Primary keys
- Foreign keys
- Functional dependencies

Avoids Anomalies in Database Operations

Normalization solves major problems called **database anomalies**:

a) Insertion Anomaly

- Problem: Cannot insert data without unrelated data.

- Example: Cannot add a new student unless they are enrolled in a course.

Normalization separates tables to allow independent insertion.

b) Update Anomaly

- Problem: Updating one record requires multiple changes.
- Risk: Some records may remain outdated.

Normalization ensures single-point update.

c) Deletion Anomaly

- Problem: Deleting one record unintentionally removes important data.
- Example: Deleting last student record may delete course info.

Normalization prevents loss of unrelated data.

Better Query Performance (in many cases)

Smaller, well-structured tables allow:

- Faster searches
- Efficient indexing
- Better query optimization

Simplifies Database Maintenance

A normalized database is easier to:

- Understand
- Modify
- Expand

Eliminates Data Duplication

Each fact is stored only once, reducing unnecessary repetition.

Enhances Data Security

Since data is divided into multiple related tables, access control can be applied more effectively.

Improves Logical Data Organization

Data is structured based on relationships, making it more meaningful and logically grouped.

Supports Scalability

Normalized databases can easily grow without major redesign.

14. Develop an ER to relational mapping for a system with weak entity dependent related to student, including all constraints

ER to Relational Mapping (Weak Entity dependent on Student)

Let's consider a system where:

- **Student** is a strong entity
- **Dependent** (e.g., *Family Member / Dependent Person*) is a **weak entity** that depends on Student

ER Model Description

Strong Entity: STUDENT

Attributes:

- Student_ID (Primary Key)
- Name
- Age
- Department

Weak Entity: DEPENDENT

Attributes:

- Dependent_Name (Partial Key)
- Relationship
- Age

Relationship: HAS_DEPENDENT

- One Student can have many Dependents

- Each Dependent must belong to exactly one Student
- This is an **identifying relationship**

Constraints in ER Model

Entity Constraints

- Student_ID → Primary Key (unique, not null)
- Dependent_Name → Partial Key (unique only within a Student)

Referential Constraint

- Dependent cannot exist without Student

Participation Constraint

- Total participation of DEPENDENT in HAS_DEPENDENT relationship
- Every dependent must be linked to a student

Cardinality Constraint

- Student : Dependent = 1 : N

ER to Relational Mapping Rules

Step 1: Convert Strong Entity (Student)

STUDENT(Student_ID, Name, Age, Department)

Primary Key: Student_ID

Convert Weak Entity (Dependent)

For weak entity, we include:

- Primary key of strong entity (Student_ID)
- Partial key (Dependent_Name)

DEPENDENT(Student_ID, Dependent_Name, Relationship, Age)

Primary Key: (Student_ID, Dependent_Name)

Foreign Key: Student_ID references STUDENT(Student_ID)

Final Relational Schema

STUDENT Table

Attribute	Type	Constraint
Student_ID	INT	PRIMARY KEY
Name	VARCHAR	NOT NULL
Age	INT	
Department	VARCHAR	

DEPENDENT Table

Attribute	Type	Constraint
Student_ID	INT	FOREIGN KEY → STUDENT(Student_ID)
Dependent_Name	VARCHAR	PARTIAL KEY
Relationship	VARCHAR	NOT NULL
Age	INT	

Primary Key: (Student_ID, Dependent_Name)

STUDENT (Student_ID) ————< HAS_DEPENDENT >———— DEPENDENT

|

(Dependent_Name, Relationship, Age)

15. A database contains relations, Client(CID, Name), Orders(OID,CID,Amount). Analyse how relational algebra operations can be combined to find the clients who have placed orders above 5000

We are given two relations:

- **Client(CID, Name)**
- **Orders(OID, CID, Amount)**

We need to find **clients who have placed orders with Amount > 5000** using **Relational Algebra**.

Select Orders with Amount > 5000

First, filter only those orders that satisfy the condition:

$\sigma_{Amount>5000}(Orders)$

This gives a relation containing only high-value orders.

Join with Client Relation

Next, we need to connect these filtered orders with client details using **CID (common attribute)**:

$Client \bowtie \sigma_{Amount>5000}(Orders)$

This is an **equijoin/natural join on CID**, combining client information with matching orders.

Project Required Attributes

Finally, we extract only the required details (client information). Usually, we want **CID and Name**:

$\pi_{CID,Name}(Client \bowtie \sigma_{Amount>5000}(Orders))$

Final Relational Algebra Expression

$\pi_{CID,Name}(Client \bowtie \sigma_{Amount>5000}(Orders))$

Combined Operations

Selection (σ)

- Filters orders where **Amount > 5000**
- Reduces unnecessary tuples early

Join (⋈)

- Combines Client and filtered Orders using **CID**
- Links customers with their qualifying orders

Projection (π)

- Extracts only relevant attributes (CID, Name)
- Removes unnecessary columns like OID and Amount

16.Compare static hashing and dynamic hashing?

Static Hashing

Static hashing uses a fixed number of buckets (memory locations) where records are stored using a hash function.

Working

- A hash function (e.g., h(k) = k mod n) maps a key to a bucket.
- The number of buckets (n) is fixed at the time of file creation.
- If multiple keys map to the same bucket → collision occurs.

Collision Handling Techniques

- Chaining (linked list of records)
- Open addressing
- Overflow buckets

Advantages

- Simple and easy to implement
- Fast access when dataset size is stable
- Low overhead

Disadvantages

- Poor performance when data grows

- Causes overflow chains, increasing search time
- Difficult to resize (requires rehashing entire file)

Dynamic Hashing

Dynamic hashing allows the number of buckets to grow or shrink dynamically as the database size changes.

Working

- Uses advanced techniques like:
  - Extendible Hashing
  - Linear Hashing
- Buckets are split when they overflow
- A directory (in extendible hashing) or gradual expansion (in linear hashing) manages growth

A. Extendible Hashing

- Uses a directory of pointers
- Hash function gives a binary value
- Uses global depth and local depth
- Buckets split when full

B. Linear Hashing

- No directory needed
- Buckets split gradually in sequence
- Uses a split pointer

Advantages

- Handles growing and shrinking data efficiently
- Minimizes overflow chains
- Better performance for large and dynamic datasets

Disadvantages

- More complex to implement
- Extra memory overhead (directory in extendible hashing)
- Slightly slower than static hashing for small datasets

Feature	Static Hashing	Dynamic Hashing
Bucket Size	Fixed	Grows/Shrinks dynamically
Flexibility	Low	High
Performance	Degrades with growth	Stable even with growth
Collision Handling	Overflow chains	Bucket splitting
Space Utilization	Poor (may waste or overflow)	Efficient
Complexity	Simple	Complex
Rehashing	Required when resizing	Not required (handled automatically)

17. Consider the relation Supply_part(Supply_ID,Supply_Name,Part_ID,Part_Name,Amount) Functional Dependencies are Supply_ID→Supply_Name,Part_ID→Part_Name, (Supply_ID,Part_ID)→Amount . Identify the highest Normal form

Understand the Relation

Supply_part(Supply_ID, Supply_Name, Part_ID, Part_Name, Amount)

This means:

- Each row tells us which supplier supplies which part and in what amount

Given Functional Dependencies (FDs)

1. Supply_ID → Supply_Name
   A supplier ID uniquely determines the supplier name
2. Part_ID → Part_Name
   A part ID uniquely determines the part name
3. (Supply_ID, Part_ID) → Amount
   The combination of supplier and part determines the amount supplied

Find Candidate Key

We check which attributes determine all other attributes.
Try:
(Supply_ID, Part_ID)
From this:

- Supply_ID → Supply_Name
- Part_ID → Part_Name
- (Supply_ID, Part_ID) → Amount

So we get:
(Supply_ID, Part_ID) → all attributes
Candidate Key = (Supply_ID, Part_ID)

Check First Normal Form (1NF)

- All attributes must be atomic (no repeating groups)

Check:

- Supply_ID → single value
- Supply_Name → single value
- Part_ID → single value
- Part_Name → single value
- Amount → single value

Relation is in 1NF

Check Second Normal Form (2NF)

- Must be in 1NF
- No partial dependency
  Non-key attributes must depend on whole composite key, not part of it

Identify key:
Composite key = (Supply_ID, Part_ID)
Check dependencies one by one:
FD1: Supply_ID → Supply_Name

- Supply_Name depends only on Supply_ID
- But the key is (Supply_ID, Part_ID)

This is a partial dependency
FD2: Part_ID → Part_Name

- Part_Name depends only on Part_ID
- Not on full key

Another partial dependency
FD3: (Supply_ID, Part_ID) → Amount

- Amount depends on full key

This is correct
Conclusion:
Because of partial dependencies:
Relation is NOT in 2NF

Check Third Normal Form (3NF)

- Must be in 2NF
- No transitive dependency

Since it already fails 2NF:

It cannot be in 3NF
Check BCNF
For every FD:
Left side must be a super key
Check:

- Supply_ID → Supply_Name
  Supply_ID is not a super key
- Part_ID → Part_Name
  Part_ID is not a super key

Not in BCNF
Final Answer
The highest normal form is 1NF

18.Compare homogeneous and heterogeneous databases with examples?

Homogeneous Databases

A homogeneous database system is one where all databases use the same DBMS software, data model, and schema structure.
Key Characteristics

- Same database software (e.g., all use MySQL or Oracle Database)
- Same data model (usually relational)
- Uniform schema and query language (like SQL)
- Easier to manage and integrate

Example
A multinational company uses MySQL in all its branches:

- India branch stores employee data in MySQL
- USA branch also uses MySQL with identical schema
- Data can be easily shared and queried across locations

Since everything is standardized, there's no need for data conversion or translation.
Advantages

- Simple data integration
- Faster query processing
- Easy maintenance and security
- High consistency

Disadvantages

- Less flexibility (all systems must follow the same structure)
- Difficult to adopt new technologies in one part without affecting others

Heterogeneous Databases

A heterogeneous database system consists of different types of databases using different DBMS software, models, or schemas, but connected together.
Key Characteristics

- Different DBMSs (e.g., MySQL, MongoDB, Microsoft SQL Server)
- Different data models (relational, document, graph, etc.)
- Schema differences
- Requires middleware or translation layer

Example
An e-commerce company uses:

- Customer data in MySQL
- Product catalog in MongoDB

- Sales analytics in Microsoft SQL Server

These systems are connected, but since they differ, data conversion and integration tools are required.

Advantages

- High flexibility (use best DB for each task)
- Easier to integrate legacy systems
- Scalable and adaptable

Disadvantages

- Complex integration
- Slower query performance due to translation
- Data inconsistency risks
- Higher maintenance cost

Feature	Homogeneous DB	Heterogeneous DB
DBMS Type	Same	Different
Data Model	Uniform	Mixed
Integration	Easy	Complex
Flexibility	Low	High
Maintenance	Simple	Complicated
Example	All branches using MySQL	Mix of MySQL, MongoDB, SQL Server

19.What is a deadlock? Explain how it can be handled?

Deadlock (in Database Systems)

A deadlock in a database occurs when two or more transactions are permanently blocked because each is waiting for a resource locked by another transaction. In DBMS, this usually happens due to locking mechanisms used to maintain consistency.

Database-Oriented Example

Consider a banking database:

- Transaction T1 transfers money from Account A → B
- Transaction T2 transfers money from Account B → A

Step-by-step:

1. T1 locks Account A
2. T2 locks Account B
3. T1 tries to lock Account B → must wait
4. T2 tries to lock Account A → must wait

Now both transactions are waiting for each other

Deadlock occurs

Deadlocks are mainly caused by:

- Simultaneous transactions
- Lock-based concurrency control
- Improper ordering of resource access

Deadlocks Are Handled in Databases

- Deadlock Prevention (Before It Happens)

  The DBMS ensures that at least one deadlock condition is eliminated.
  In databases:
  - Transactions must lock resources in a fixed order
  - Or request all required locks at once
    Example:

  If all transactions must lock Account A first, then B, deadlock won't occur.

Deadlock Avoidance

The DBMS checks whether granting a lock will lead to a deadlock.

- Uses algorithms like Banker's Algorithm
- Keeps the system in a safe state

Example:

If T1 requesting a lock may cause circular waiting, DBMS delays it.

Deadlock Detection and Recovery (Most Common in DBMS)

DBMS allows deadlocks but detects and resolves them

Detection:

- Uses a Wait-For Graph
- If a cycle exists → deadlock detected

Recovery:

- Abort one transaction (called the *victim*)
- Rollback its changes
- Release locks

Example:

In the banking case:

- DBMS detects T1 ↔ T2 cycle
- It aborts T2
- Releases Account B lock
- T1 completes successfully

20.Transactions:

a. **T1 locks A→ requests B**
b. **T2 locks B→ requests A**
   i. **How deadlock happens**
   ii. **Draw wait for graph**
   iii. **Explain deadlock prevention mechanism**

Given Transactions

- T1: Locks A → Requests B
- T2: Locks B → Requests A

(i) Deadlock Happens

Step-by-step execution:

1. T1 starts
   - Locks resource A
2. T2 starts
   - Locks resource B
3. T1 requests B
   - But B is already locked by T2
   - So, T1 waits
4. T2 requests A
   - But A is already locked by T1
   - So, T2 waits

Final Situation:

- T1 is waiting for T2
- T2 is waiting for T1

Neither can proceed → Deadlock occurs

Deadlock happens when there is a circular waiting condition

(ii)Wait-For Graph (WFG)

Definition:

A Wait-For Graph shows which transaction is waiting for which other transaction.

Build the graph:

- T1 is waiting for T2 → Draw edge T1 → T2
- T2 is waiting for T1 → Draw edge T2 → T1

Graph Representation:

T1 → T2

↑ ↓

T2 ← T1

Or simply:

T1 → T2 → T1

Important Rule:

If the graph contains a cycle, then deadlock exists

Here:

- Cycle = T1 → T2 → T1

So, deadlock is confirmed

(iii) Deadlock Prevention Mechanisms

Deadlock occurs due to 4 necessary conditions:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

Prevention = Break any one condition

1. Eliminate Hold and Wait

Idea:

A transaction must request all resources at once before execution.

Example:

- T1 requests A and B together
- If both available → proceed
- Otherwise → wait

Advantage:

- No partial holding → no deadlock

Disadvantage:

- Poor resource utilization

Eliminate No Preemption

Idea:

If a transaction is waiting, take away its resources

Example:

- T1 holds A, wants B
- If B not available → T1 releases A
- T2 can proceed

Advantage:

- Avoids deadlock

Disadvantage:

- Wastes work (rollback)

Eliminate Circular Wait (Most Important)

Idea:

Assign ordering to resources

Example:

A = 1, B = 2

Rule:
Transactions must request resources in increasing order
Apply to problem:

- T1: Lock A → then B
- T2: Must also lock A before B

So:

- T2 cannot lock B first ❌

Circular wait is avoided
Advantage:

- Simple and effective

Timestamp-based Prevention
Two famous techniques:
(a) Wait-Die Scheme

- Older transaction → waits
- Younger transaction → aborted

Example:

- T1 (older), T2 (younger)

Case:

- T1 requests resource from T2 → T1 waits
- T2 requests resource from T1 → T2 aborts

Deadlock avoided
(b) Wound-Wait Scheme

- Older transaction → aborts younger
- Younger → waits

Example:

- T1 (older), T2 (younger)

Case:

- T1 requests B (held by T2) → T2 aborted
- T2 requests A → waits

No cycle forms

21. Explain in detail about the basic inference rules that are used to derive the functional dependency

22. Explain the Wait-Die and Wound-Wait timestamp based schemes.

23.What are the different file organization methods? Explain any two?

File organization methods in database systems define **how records are physically stored on disk**. The choice of file organization directly affects **search speed, insertion, deletion, and overall performance** in a DBMS.
Common file organization methods include:

- Heap (Unordered) File Organization
- Sequential (Ordered) File Organization
- Hash File Organization

- Clustered File Organization

1. Heap File Organization (Unordered File Organization)

In **Heap File Organization**, records are stored **randomly in the file without any order**. When new records are inserted, they are simply placed in the first available space.

Working

- No sorting based on key attributes
- New records are inserted at the end or any free space
- Searching requires **linear search**

Example

Consider a Student table:

Roll No	Name
105	Ravi
101	Anu
110	John

Records are stored in **random order physically on disk**, not sorted.

Advantages

- Very simple structure
- Fast insertion (no sorting required)
- Suitable for small databases or temporary storage

Disadvantages

- Slow searching (linear scan)
- Deletion and update operations may be inefficient

2. Sequential (Ordered) File Organization

In **Sequential File Organization**, records are stored in **sorted order based on a key attribute (usually primary key)**.

Working

- Records are stored in ascending or descending order
- Searching can use **binary search or indexed access**
- Insertion requires shifting or reorganization

Example

Student records sorted by Roll No

Roll No	Name
101	Anu
105	Ravi
110	John

Data is physically stored in **sorted order**

Advantages

- Fast searching (especially for range queries)
- Efficient for reports and batch processing
- Good for ordered retrieval

Disadvantages

- Slow insertion (requires shifting or reorganization)
- Deletion is costly

Heap and Sequential File Organization

Feature	Heap File	Sequential File
Order	Unordered	Sorted
Insertion	Fast	Slow
Searching	Slow (linear)	Fast (binary/range)
Best Use	Insert-heavy systems	Read-heavy systems

24. Explain Strict Two Phase locking and analyze how its better than the basic one

Strict Two-Phase Locking (Strict 2PL) is an enhanced version of the basic **Two-Phase Locking (2PL)** protocol used in database concurrency control. In the basic 2PL protocol, a transaction follows two phases: a growing phase, where it acquires all the required locks, and a shrinking phase, where it releases locks and cannot acquire any new ones. While this ensures serializability, it still allows certain issues like cascading rollbacks because a transaction may release locks before it fully completes, allowing other transactions to read uncommitted data.

In **Strict Two-Phase Locking**, the rule is stricter: a transaction must hold all its exclusive (write) locks until it either commits or aborts. In other words, **all write locks are released only after the transaction finishes successfully**. Read locks may be released earlier in some variations, but write locks are strictly held until the end. This ensures that no other transaction can read or write data that has been modified but not yet committed.

For example, consider two transactions T1 and T2 accessing a bank account balance. If T1 updates the balance but has not yet committed, Strict 2PL ensures that T2 cannot read this uncommitted value. T2 must wait until T1 either commits or rolls back. This prevents inconsistent or "dirty" reads and avoids cascading rollbacks, where one transaction failure forces others to roll back.

Strict 2PL is better than basic 2PL because it provides **stronger data consistency and recoverability guarantees**. It ensures that the schedule is not only serializable but also **strictly recoverable**, meaning committed data is always consistent and no transaction reads uncommitted changes. This simplifies database recovery mechanisms because the system does not need to undo multiple dependent transactions in case of failure.

However, the trade-off is that Strict 2PL can reduce concurrency because transactions hold locks for a longer time, which may increase waiting time. Despite this, it is widely used in real database systems like **Oracle Database** and **MySQL** because the benefit of safety and consistency outweighs the performance cost in most applications.

In summary, Strict 2PL improves the basic 2PL protocol by ensuring higher safety, preventing dirty reads, avoiding cascading rollbacks, and providing better reliability in multi-user database environments, making it more suitable for real-world database systems

25. Explain how RAID improves the performance and data loss

26. Describe CAP Theorem and explain how it applies to distributed databases?

The **CAP Theorem** is a fundamental principle in distributed database systems that explains the trade-offs involved when designing systems that operate across multiple servers or networked nodes. It was introduced by Eric Brewer and states that a distributed system can

provide only **two out of the following three guarantees at the same time: Consistency, Availability, and Partition Tolerance**.

Consistency (C) means that all nodes in the system show the same data at the same time. In other words, after a write operation, any read request will return the most recent value. For example, in a banking system, if a customer transfers money from Account A to Account B, all servers should immediately reflect the updated balance.

Availability (A) means that every request (read or write) receives a response, even if some nodes are down. The system remains operational at all times. For example, an online shopping website should still allow users to browse products even if some database servers are temporarily unavailable.

Partition Tolerance (P) means that the system continues to function even if network failures occur that split the system into isolated parts. This is very important in real-world distributed systems because network failures are unavoidable. For example, if a cloud database has servers in India and the USA and the connection between them fails, both sides should still operate independently.

The CAP theorem states that in the presence of a network partition, a distributed system must choose between consistency and availability. This means that when a partition occurs, the system cannot guarantee both perfect consistency and full availability at the same time.

For example, consider a distributed database like **MongoDB** or **Cassandra**. In some configurations, Cassandra prioritizes availability and partition tolerance (AP system). This means even if some nodes cannot communicate, the system will still respond to requests, but the data might not be immediately consistent across all nodes. On the other hand, systems like traditional relational databases or strongly consistent distributed systems prioritize consistency and partition tolerance (CP system), meaning that during a network failure, some requests may be rejected to ensure all nodes remain consistent.

In real-world applications, this trade-off is very important. For example, social media platforms may prefer availability so that users can always post and view content, even if updates are slightly delayed across servers. In contrast, banking systems prefer consistency over availability because incorrect or inconsistent financial data cannot be tolerated.

Thus, the CAP theorem helps database designers decide the appropriate balance based on application requirements. It shows that in distributed databases, it is impossible to achieve all three properties simultaneously, and system design must prioritize two depending on the use case

27. What is distributed transactions? Explain briefly?

28. Explain database recovery techniques and the role of the transaction log?

29.How B tree and B+ tree play a major role in indexing. Explain in detail

Indexing in Databases

Indexing is a technique that improves the speed of data access. Instead of searching every row (full table scan), the DBMS uses a structured index (like a tree) to quickly locate records. Most modern DBMS like MySQL, Oracle Database, and Microsoft SQL Server use B+ trees for indexing.

B-Tree

A B-tree is a self-balancing tree data structure that maintains sorted data and allows searches, insertions, and deletions in logarithmic time.

Key Features

- Nodes can have multiple keys and children
- Data can be stored in both internal and leaf nodes
- Tree remains balanced (all leaves at same level)

Example of B-Tree Index

Suppose we store student IDs:

10, 20, 30, 40, 50

A B-tree might look like:

```
      [30]
     /   \
 [10,20]   [40,50]
```

To search 40:

- Compare with 30 → go right
- Find in node [40,50]

Faster than scanning all values

B+ Tree

A B+ tree is an advanced version of B-tree, specially designed for database indexing.

Key Features

- Only leaf nodes store actual data (or pointers to data)
- Internal nodes store only keys (for navigation)
- Leaf nodes are linked (like a linked list)
- Better for range queries

Example of B+ Tree

For same values:

```
       [30]
      /   \
    [20]   [40]
   / \    / \
[10,20][30][40,50]
```

Leaf nodes (linked):

[10,20] → [30] → [40,50]

To search 40:

- Navigate via internal nodes
- Reach leaf → get data

To find range (20–50):

- Start at 20
- Traverse linked leaves → very fast

Role of B-Tree and B+ Tree in Indexing

1. Fast Searching
   - Time complexity: O(log n)
   - Efficient even for millions of records
2. Efficient Disk Access
   - Databases store data on disk
   - Trees reduce number of disk reads

- Nodes are sized to match disk blocks

3. Dynamic Growth
   - Automatically adjust (split/merge nodes)
   - No need to reorganize entire database

4. Support for Range Queries (B+ Tree Advantage)

Example:
SELECT * FROM students WHERE id BETWEEN 20 AND 50;
B+ tree quickly traverses linked leaves

5. Ordered Data Access
   - Data is always sorted
   - Helps in queries with ORDER BY, GROUP BY

Feature	B-Tree	B+ Tree
Data Storage	Internal + Leaf nodes	Only Leaf nodes
Search Speed	Slightly slower	Faster
Range Queries	Less efficient	Very efficient
Leaf Linking	No	Yes
Usage in DBMS	Rare	Widely used

30.List out how failure occurs during the failure of primary nodes with the help of replica set architecture

In advanced DBMS—especially in distributed systems like MongoDB—a replica set architecture is used to ensure high availability and fault tolerance.
A replica set consists of:
- Primary node → handles all write operations
- Secondary nodes → replicate data from the primary
- (Optional) Arbiter → participates in elections

What Happens When the Primary Node Fails?

When the primary node fails, the system doesn't stop. Instead, the replica set automatically recovers through a process called failover.

Steps of Failure Handling in Replica Set Architecture

1. Failure Detection
- Secondary nodes continuously monitor the primary using heartbeat signals
- If the primary does not respond within a timeout → it is considered down

Example:
Primary crashes due to power failure or network issue

2.Election Process Starts
- Secondary nodes initiate an election to choose a new primary
- Each node votes

Conditions for election:
- Node must have latest data (or close to it)
- Must be eligible (not lagging too much)

3. New Primary is Elected
- Node with majority votes becomes the new primary
- Other nodes become secondaries

This ensures consistency and avoids split-brain problem

4. Automatic Failover
- The newly elected primary starts handling:
  - Write operations
  - Client requests

Applications reconnect automatically (driver-level support)

5. Old Primary Recovery

- When the failed primary comes back:
  - It does not immediately become primary
  - It syncs data from the new primary
  - Becomes a secondary node

Example Scenario

Replica Set:

- Node A → Primary
- Node B → Secondary
- Node C → Secondary

Failure:

- Node A crashes

Process:

1. B and C detect failure
2. Election begins
3. Node B becomes new primary
4. System continues working

No manual intervention needed

Types of Failures in Primary Node

1. Hardware Failure
   - Disk crash, power outage
2. Network Failure
   - Node becomes unreachable
3. Software Crash
   - DBMS process stops
4. Maintenance Shutdown
   - Planned restart

Challenges During Failover

- Temporary unavailability (few seconds)
- Rollback possibility (if old primary had unreplicated writes)
- Election delay

Advantages of Replica Set Handling

- High availability
- Automatic recovery
- Data redundancy
- Fault tolerance

31. Given a relation R(A, B, C, D) with functional dependencies A → BC and B → C, find all the candidate keys of the relation.

Given

Relation:

R(A, B, C, D)

Functional Dependencies (FDs):

1. **A → B, C**
2. **B → C**

Step 1: Understand the Goal

We need to find **candidate keys**.

A **candidate key** is:

- A **minimal set of attributes**

- That can **determine all attributes** in the relation (A, B, C, D)

Step 2: Key Observation

Look at attributes on the **right-hand side (RHS)** of FDs:

- From A → BC → RHS has **B, C**
- From B → C → RHS has **C**

Attributes appearing on RHS = **B, C**
Attribute **NOT on RHS = A, D**
Important rule:
Attributes **not appearing on RHS must be part of every candidate key**
So, **A and D must be included**

The **closure of a set of attributes X** (written as $\mathbf{X^+}$) is:
All attributes that can be functionally determined from X

Step 3: Check Closure of (A, D)

Now compute $\mathbf{(A, D)^+}$ (closure of AD)
Start with:
$AD^+ = \{A, D\}$
Apply FDs:

1. A → B, C
   Add B and C:

$AD^+ = \{A, B, C, D\}$

2. B → C
   (C already included, no change)

Result:
$AD^+ = \{A, B, C, D\}$
This covers **all attributes**
So, **AD is a superkey**
Step 4: Check Minimality

Now check if it's **minimal**:
Remove A:

- $D^+ = \{D\}$ (cannot determine all)

Remove D:

- $A^+ = \{A, B, C\}$ (missing D)

Neither A nor D can be removed
So, **AD is minimal → Candidate Key**
Step 5: Check for Other Candidate Keys

Try other combinations:
1. A alone:

- $A^+ = \{A, B, C\}$ (missing D)

2. B + D:

- $B^+ = \{B, C\}$
- $BD^+ = \{B, C, D\}$ (missing A)

3. A + B + D:

- Not minimal (since AD already works)

No other minimal combinations possible
Final Answer

Candidate Key(s):

AD

32. Design a DTD for the XML related to a book explain in detail about each items

A **DTD (Document Type Definition)** defines the **structure, elements, and rules** of an XML document. It ensures that an XML document follows a proper format and contains only allowed elements and attributes.

Sample XML Structure for a Book

Before writing DTD, we define how our XML might look:

```
<Book>
   <Title>Database Systems</Title>
   <Author>Elmasri</Author>
   <Publisher>Pearson</Publisher>
   <Year>2024</Year>
   <Price>500</Price>
</Book>
```

DTD for Book XML

```
<!ELEMENT Book (Title, Author, Publisher, Year, Price)>
<!ELEMENT Title (#PCDATA)>
<!ELEMENT Author (#PCDATA)>
<!ELEMENT Publisher (#PCDATA)>
<!ELEMENT Year (#PCDATA)>
<!ELEMENT Price (#PCDATA)>
```

Explanation of Each Part

(1) <!ELEMENT Book ...>

<!ELEMENT Book (Title, Author, Publisher, Year, Price)>

This defines the **root element Book**

Meaning:

- Book must contain exactly these child elements:
  - Title
  - Author
  - Publisher
  - Year
  - Price

Order is important in DTD:

- Elements must appear in the same sequence

(2) Title Element

<!ELEMENT Title (#PCDATA)>

#PCDATA means:

- Parsed Character Data (text content)

Meaning:

- Title contains only text (no child elements)

Example:

<Title>Database Systems</Title>

(3) Author Element

<!ELEMENT Author (#PCDATA)>

Stores author name as plain text

Example:
<Author>Elmasri</Author>

(4) Publisher Element

<!ELEMENT Publisher (#PCDATA)>
Stores publisher name
Example:
<Publisher>Pearson</Publisher>

(5) Year Element

<!ELEMENT Year (#PCDATA)>
Stores publication year
Example:
<Year>2024</Year>

(6) Price Element

<!ELEMENT Price (#PCDATA)>
Stores book price as text (can be validated separately)
Example:
<Price>500</Price>

Complete XML with DTD Reference

```
<!DOCTYPE Book SYSTEM "book.dtd">

<Book>
  <Title>Database Systems</Title>
  <Author>Elmasri</Author>
  <Publisher>Pearson</Publisher>
  <Year>2024</Year>
  <Price>500</Price>
</Book>
```

33. Describe the architecture of a database system with a neat diagram?

34. Describe about the ER to Relational Mapping with good examples

35. Explain the Entity-Relationship (E-R) model and its components?

36. Describe the process of normalization up to Third Normal Form (3NF) with suitable examples?

37. A file contains 10000 records and has no index.
    a. How many blocks needed to perform linear search
    b. If an index is created on the file compare the number of blocks accesses required for searching with and without the index

Given

- Number of records = 10,000
- No index initially
- Assume 1 record per block (since block size not given)

Total blocks = 10,000

(a) Blocks Needed for Linear Search

In linear search, records are checked one by one.

Cases:

- Best case → record found in first block
  1 block access
- Worst case → record is last or not present
  10,000 block accesses
- Average case → record in middle
  1000/2=5000

Average = 5000 block accesses

Final Answer (a):

- Best case = 1
- Worst case = 10,000
- Average case = 5,000

(b) With Index vs Without Index

Without Index

- Linear search
  Average = 5000 block accesses

With Index

Assume:

- Index is multi-level (B+ tree)
- Typical height = 3 levels (Root → Intermediate → Leaf)

Steps:

1. Root block → 1 access
2. Intermediate block → 1 access
3. Leaf block → 1 access
4. Data block → 1 access

Total = 4 block accesses

Final Answer (b):

- Without index = 5000 block accesses (average)
- With index ≈ 4 block accesses

38. List out the various levels of RAID and explain in detail

39. Consider a relation R(A, B, C, D) with functional dependencies A → BC and B → C.

a. Find the candidate key(s).

b. Determine whether the relation is in 3NF and BCNF.

c. If not, decompose it into relations that satisfy BCNF.

Given

Relation:

R(A, B, C, D)

Functional Dependencies:

1. **A → BC**
2. **B → C**

(a) Find Candidate Key(s)

Step 1: Find attribute closure
We try to find which attribute set gives all attributes (A, B, C, D).
Check A^+
Start:
$A^+ = \{A\}$
Apply A → BC:
$A^+ = \{A, B, C\}$
Now we cannot get D.
So A is NOT a candidate key.
Try AD^+
Start:
$AD^+ = \{A, D\}$
Apply A → BC:
$AD^+ = \{A, B, C, D\}$
All attributes obtained

Check minimality
- Remove A → $D^+ = \{D\}$ ❌
- Remove D → $A^+ = \{A, B, C\}$ ❌

Both needed → minimal

Final Answer (a)
Candidate Key = AD
(b) Check 3NF and BCNF

Step 1: Prime attributes
- Candidate key = **AD**
- Prime attributes = A, D
- Non-prime attributes = B, C

Step 2: Check BCNF condition
BCNF rule:
For every FD X → Y, X must be a super key

Check FD1: A → BC
- $A^+ = \{A, B, C\}$ (not super key)

Violates BCNF
Check FD2: B → C
- $B^+ = \{B, C\}$ (not super key)

Violates BCNF

So relation is NOT in BCNF

Step 3: Check 3NF condition
3NF rule:
For every FD X → Y:
- X is super key OR
- Y is prime attribute

FD1: A → BC

- A not super key
- B, C are non-prime

Violates 3NF

FD2: B → C

- B not super key
- C is non-prime

Violates 3NF

So relation is NOT in 3NF either

Final Answer (b)

- Not in 3NF
- Not in BCNF

(c) BCNF Decomposition

We decompose using violating dependencies.

Step 1: Use A → BC

Decompose R into:

R1(A, B, C)

R2(A, D)

Step 2: Check R1(A, B, C)

FDs:

- A → BC
- B → C

Candidate key in R1:

- A^+ = ABC → A is key

So A is super key

R1 is in BCNF

Step 3: Check R2(A, D)

No FD except A, D

- Candidate key = AD
  BCNF satisfied

Final BCNF Decomposition

R1(A, B, C)

R2(A, D)

Final Summary

(a) Candidate Key:

AD

(b) Normal Form:

Not in 3NF

Not in BCNF

(c) BCNF Decomposition:

R1(A, B, C)

R2(A, D)

40. Describe the object identity and reference types in SQL with example and also make a comparison with relational database

Object Identity and Reference Types in SQL (Object-Relational Concept)

In advanced databases like **Oracle Database**, SQL is extended to support **object-oriented features**, leading to concepts like **object identity (OID)** and **reference types (REF)**.

These are mainly used in **Object-Relational DBMS (ORDBMS)**.

1. Object Identity (OID)

Object Identity (OID) is a **unique identifier assigned to each object**, independent of its attribute values.

Even if two objects have the same data, they are still different because they have different identities.

Example

Let's define an object type:

```
CREATE TYPE Student AS OBJECT (
   ID INT,
   Name VARCHAR(20)
);
```

Now:

Student1 = (1, "Amit")

Student2 = (1, "Amit")

Even though data is same:

- Student1 ≠ Student2
- Because OIDs are different

Identity is not based on values, but on **internal system-generated ID**

Real-Life Analogy

- Two identical twins
  Same appearance (data)
  Still different individuals (identity)

2. Reference Types (REF)

A **reference type (REF)** is a **pointer-like reference to an object stored in another table or object type**.

Instead of copying data, we store a **reference (address-like link)**.

Example

```
CREATE TYPE Department AS OBJECT (
   Dept_ID INT,
   Dept_Name VARCHAR(20)
);
```

Now create Employee referencing Department:

```
CREATE TYPE Employee AS OBJECT (
   Emp_ID INT,
   Name VARCHAR(20),
   Dept REF Department
);
```

Usage Example

Dept1 = Department(10, "IT")

Emp1 = Employee(1, "John", REF Dept1)

Here:

- Employee does NOT store full department data
- It stores a **reference (pointer) to Department object**

Object Model vs Relational Model

Feature	Object-Relational Model (ORDBMS)	Relational Model
Identity	Object Identity (OID)	Primary Key
Relationships	REF (pointer/reference)	Foreign Key
Data Storage	Objects	Tables (rows & columns)
Data Access	Direct object navigation	Join operations
Redundancy	Reduced (uses references)	Can be higher
Example System	Oracle Database	MySQL

41. What is functional dependencies, inference rules and minimal set of functional dependencies?

42. Describe the role of Concurrency Control with Locking Methods?

43.The schedule :

a. **T1: Lock-X(A),Read(A),Write(A),Unlock(A)**
b. **T2: Lock-X(A),Read(A),Write(A),Unlock(A)**
    i. **The schedule follows 2PL or Not**
    ii. **Check if it is serializable**

Given Schedule

Transaction T1:

Lock-X(A)
Read(A)
Write(A)
Unlock(A)

Transaction T2:

Lock-X(A)
Read(A)
Write(A)
Unlock(A)

Both transactions are accessing the **same data item A exclusively (X-lock)**.

(i) Does the schedule follow 2PL?

2PL (Two-Phase Locking)?

A transaction follows **Two-Phase Locking (2PL)** if:

- **Phase 1 (Growing phase):** It can only acquire locks
- **Phase 2 (Shrinking phase):** It can only release locks
- Once a lock is released, **no new lock can be acquired**

Check T1

T1 sequence:

- Lock-X(A) → acquiring lock
- Read, Write → operations
- Unlock(A) → releasing lock

No new lock after unlock

T1 follows 2PL

Check T2

T2 sequence:

- Lock-X(A) → acquire
- Read, Write → operations
- Unlock(A) → release

Same pattern

T2 also follows 2PL

Conclusion (i):

Yes, the schedule follows 2PL

(both T1 and T2 individually satisfy 2PL rules)

Simple Example for 2PL

Think like this:

- You **take a key (lock)** before entering a room
- Do all work inside
- Then **return the key (unlock)**
- You cannot take a new key after returning it

(ii) Is the schedule Serializable?

Serializability

A schedule is **Conflict Serializability** if:

- It produces the same result as some **serial execution** of transactions
- No conflicting operations break consistency

Step 1: Identify Conflict

Both transactions access:

- Same data item: **A**
- Both perform:
  - Read(A)
  - Write(A)

So conflicts exist between T1 and T2

Step 2: Possible Execution Order

Since both use **X-lock**, only one can execute at a time.

Case 1: T1 executes first

T1 → T2

Result:

- T1 completes fully
- Then T2 starts

Case 2: T2 executes first

T2 → T1

Result:

- T2 completes fully
- Then T1 starts

Step 3: Check Equivalence

Both outcomes are identical to a **serial schedule**.

No interleaving is actually possible due to X-lock

Conclusion (ii):

Yes, the schedule is serializable

It is **conflict-serializable**

Equivalent to a serial order (T1 → T2 or T2 → T1)

Simple Example for Serializability

Imagine:

- Only one person can use a computer at a time
- Either T1 uses it fully, then T2
- OR T2 uses it fully, then T1

No mixed execution → safe and consistent

Final Answer Summary

(i) 2PL?

Yes, both T1 and T2 follow Two-Phase Locking

(ii) Serializable?

Yes, it is conflict-serializable

Equivalent to a serial schedule:

- T1 → T2 OR
- T2 → T1

44. Consider the relation
    Employee(Emp_ID,Emp_Name,Dept_ID,Dept_Name,Trainer_ID,Trainer_Phone)
    Functional Dependencies are (Emp_ID→Emp_Name, Dept_ID→Dept_Name, Trainer, Trainer→Trainer_Phone).Normalize the relation upto 3NF

Given Relation

Employee(Emp_ID, Emp_Name, Dept_ID, Dept_Name, Trainer_ID, Trainer_Phone)

Given Functional Dependencies (FDs)

1. **Emp_ID → Emp_Name**
2. **Dept_ID → Dept_Name**
3. **Trainer_ID → Trainer_Phone**

Step 1: Understand the Problem (Important Insight)

We observe:

- One employee belongs to a department
- Each department has a department name
- Each employee has a trainer
- Each trainer has a phone number

So there is redundancy (repetition of data) in one big table.

Step 2: Find Candidate Key

We check what uniquely identifies a record:

Attributes:

- Emp_ID gives Emp_Name
- Dept_ID gives Dept_Name
- Trainer_ID gives Trainer_Phone

But none alone gives all attributes.

So:

Candidate Key = Emp_ID

Because Emp_ID identifies each employee uniquely, and indirectly other details are linked.

Step 3: Check Normal Forms

1NF (First Normal Form)

Condition:

- Atomic values (no repeating groups)

Given relation already has atomic values.
So relation is in 1NF
2NF (Second Normal Form)

Condition:

- Must be in 1NF
- No partial dependency (non-key attribute depending on part of a composite key)

Here candidate key is single attribute (Emp_ID)
So no partial dependency exists.
So relation is in 2NF
3NF (Third Normal Form)

Condition:

- Must be in 2NF
- No transitive dependency (non-key → non-key dependency)

Identify Transitive Dependencies
From given FDs:
1. Dept_ID → Dept_Name

Dept_Name depends on Dept_ID (not Emp_ID directly)
2. Trainer_ID → Trainer_Phone

Trainer_Phone depends on Trainer_ID
Problem in Original Relation
Emp_ID → Dept_ID → Dept_Name
Emp_ID → Trainer_ID → Trainer_Phone
These are transitive dependencies

Step 4: Decompose into 3NF Tables

We split the relation to remove redundancy.
Table 1: Employee
Employee(Emp_ID, Emp_Name, Dept_ID, Trainer_ID)
Stores employee-specific data only
Table 2: Department
Department(Dept_ID, Dept_Name)
Removes dependency Dept_ID → Dept_Name
Table 3: Trainer
Trainer(Trainer_ID, Trainer_Phone)
Removes dependency Trainer_ID → Trainer_Phone

Step 5: Final 3NF Structure

1. Employee
   - Emp_ID (PK)
   - Emp_Name
   - Dept_ID (FK)
   - Trainer_ID (FK)
2. Department
   - Dept_ID (PK)
   - Dept_Name
3. Trainer
   - Trainer_ID (PK)
   - Trainer_Phone

Example to Understand

Before Normalization (Problem Table)

Emp_ID	Emp_Name	Dept_ID	Dept_Name	Trainer_ID	Trainer_Phone
E1	John	D1	IT	T1	99999
E2	Mike	D1	IT	T1	99999

Repetition:

- Dept_Name repeated
- Trainer_Phone repeated

After 3NF

Employee Table

Emp_ID	Emp_Name	Dept_ID	Trainer_ID
E1	John	D1	T1
E2	Mike	D1	T1

Department Table

Dept_ID	Dept_Name
D1	IT

Trainer Table

Trainer_ID	Trainer_Phone
T1	99999